Analyzing the Severity and Causes of Major Power Outages in the U.S.

Name(s): Samsoo Seo

Website Link: https://samsooseo.github.io/dsc259r-final-project/

In [1]:
```
import pandas as pd
import numpy as np
from pathlib import Path
import plotly.express as px
import plotly.graph_objects as go
pd.options.plotting.backend = 'plotly'
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV
# from dsc259r_utils import * # Feel free to uncomment and use this.
```

Introduction

This project conducts an open-ended investigation into the **Major Power Outages** dataset, which contains records of significant power outages in the United States from 2000 to 2016. Understanding the causes and consequences of these outages is critical for improving infrastructure resilience and public safety.

The primary focus of this project is to analyze how different environmental and operational factors—particularly **Severe Weather**—impact the restoration speed and overall severity of these events.

Primary Question: > "Do power outages caused by Severe Weather last longer on average than those caused by other reasons?"

Dataset Overview

The dataset contains **1,534 rows**, where each row represents a major power outage event. To answer the primary question and build a predictive model, the following columns are most relevant:

- `OUTAGE.DURATION` : The total time (in minutes) from the start of the outage until full restoration. (Target Variable)
- `CAUSE.CATEGORY` : The primary cause of the outage (e.g., Severe Weather, Intentional Attack).
- `CLIMATE.REGION` : The U.S. climate region where the outage occurred.
- `ANOMALY.LEVEL` : The Oceanic Niño Index (ONI) representing climate abnormalcy at the time.
- `CUSTOMERS.AFFECTED` : The number of customers who lost power during the event.

Data Cleaning and Exploratory Data Analysis

- Data Cleaning

To prepare the dataset for analysis and modeling, I performed several data cleaning steps to ensure consistency and accuracy:

- **Temporal Integration**: I combined separate date and time columns (e.g., `OUTAGE.START.DATE` and `OUTAGE.START.TIME`) into single `pd.Timestamp` objects. This allows for precise calculation of `OUTAGE.DURATION`, which is the target variable for this project.

- **Logical Validation**: I addressed data entry inconsistencies by identifying rows where the restoration time was logged before the start time. Since these negative duration values are likely manual recording errors, I treated them as missing values (`NaN`) to maintain statistical integrity.

- **Type Consistency**: Several numeric features were initially loaded as strings due to non-numeric placeholders in the raw Excel file. I coerced columns like `ANOMALY.LEVEL` and `CUSTOMERS.AFFECTED` into appropriate numeric types to prevent errors during the exploratory analysis and modeling phases.

- **Removing Metadata Rows**: I removed the first row of the raw dataset, which contained unit descriptions (e.g., "numeric", "units") rather than actual data observations, to prevent data type conflicts.

The following table shows the first few rows of the cleaned dataset, specifically focusing on the newly created temporal columns and the target variable.

In [2]:
```
# Load data, skipping metadata headers
df = pd.read_excel('outage.xlsx', skiprows=5)
```

```python
# Drop the 'Units' row and reset index
df = df.drop(index=0).reset_index(drop=True)

# Strip whitespace from column names
df.columns = df.columns.str.strip()

# Helper to merge Date/Time and convert to Timestamp
def to_timestamp(date_col, time_col):
    combined = df[date_col].astype(str) + ' ' + df[time_col].astype(str)
    # Use 'mixed' format to handle inconsistencies
    return pd.to_datetime(combined, errors='coerce', format='mixed')

# Create start and restoration timestamps
df['OUTAGE.START'] = to_timestamp('OUTAGE.START.DATE', 'OUTAGE.START.TIME')
df['OUTAGE.RESTORATION'] = to_timestamp('OUTAGE.RESTORATION.DATE', 'OUTAGE.RESTORATION.TIME')

# Calculate duration in minutes
df['OUTAGE.DURATION'] = (df['OUTAGE.RESTORATION'] - df['OUTAGE.START']).dt.total_seconds() / 60

# Handle logical errors (restoration before start)
df.loc[df['OUTAGE.DURATION'] <= 0, 'OUTAGE.DURATION'] = np.nan

# Ensure numeric columns are properly cast
cols_to_fix = ['ANOMALY.LEVEL', 'CUSTOMERS.AFFECTED', 'DEMAND.LOSS.MW']
for col in cols_to_fix:
    df[col] = pd.to_numeric(df[col], errors='coerce')

# Preview processed data
display(df[['OUTAGE.START', 'OUTAGE.RESTORATION', 'OUTAGE.DURATION', 'CAUSE.CATEGORY']].head())
```

	OUTAGE.START	OUTAGE.RESTORATION	OUTAGE.DURATION	CAUSE.CATEGORY
0	2011-07-01 17:00:00	2011-07-03 20:00:00	3060.0	severe weather
1	2014-05-11 18:38:00	2014-05-11 18:39:00	1.0	intentional attack
2	2010-10-26 20:00:00	2010-10-28 22:00:00	3000.0	severe weather
3	2012-06-19 04:30:00	2012-06-20 23:00:00	2550.0	severe weather
4	2015-07-18 02:00:00	2015-07-19 07:00:00	1740.0	severe weather

---

- Exploratory Data Analysis (EDA)

1. Distribution of Outage Durations (Fig 1)

- **Observation**: The distribution of `OUTAGE.DURATION` is heavily **right-skewed**.
- **Insight**: While most outages are resolved relatively quickly, a small number of extreme events (outliers) last for a very long time. This indicates that the **Median** is a more robust measure of central tendency than the Mean for this specific dataset.

2. Most Frequent Causes (Fig 2)

- **Observation**: **Severe Weather** is identified as the leading cause of major power outages, with **763 recorded events**.
- **Insight**: Given its high frequency and impact, analyzing weather-related outages is essential for understanding overall grid reliability and infrastructure resilience.

3. Duration by Cause (Fig 3)

- **Observation**: `Severe Weather` and `fuel supply emergency` exhibit significantly higher variability and longer durations.
- **Insight**: Different causes have unique "time signatures." Weather-related events pose the greatest risk for long-term power loss, necessitating specialized restoration strategies.

4. Deep Dive: Severe Weather Types (Fig 4)

- **Observation**: Within the 716 weather events, **Hurricanes and Winter Storms** result in significantly longer outages.
- **Insight**: The specific *type* of meteorological event is a critical predictor for restoration time. This granular information will be a key feature in the predictive modeling phase.

In [3]:
```python
# --- Preprocessing: Final trim of whitespace for consistency ---
df['CAUSE.CATEGORY'] = df['CAUSE.CATEGORY'].str.strip()

# 1. Univariate Analysis: Outage Duration Distribution
# Visualizing the overall distribution to check for skewness
fig1 = px.histogram(df, x='OUTAGE.DURATION', nbins=100,
                    title='Distribution of Outage Durations',
                    labels={'OUTAGE.DURATION': 'Duration (min)'})
```

```
fig1.show()

# 2. Univariate Analysis: Cause Category Frequency
# Identifying the most frequent causes to justify focusing on Severe Weather
cause_counts = df['CAUSE.CATEGORY'].value_counts().reset_index()
fig2 = px.bar(cause_counts, x='CAUSE.CATEGORY', y='count',
              title='Frequency of Outage Causes')
fig2.show()

# 3. Bivariate Analysis: Outage Duration by Cause (Box Plot)
# Comparing restoration times across different cause categories
fig3 = px.box(df, x='CAUSE.CATEGORY', y='OUTAGE.DURATION',
              title='Outage Duration by Cause',
              color='CAUSE.CATEGORY')
fig3.show()

# 4. Bivariate Analysis: Duration by Specific Weather Events (Filtered)
# Deep dive into 'Severe Weather' to see which specific events are most severe
weather_df = df[df['CAUSE.CATEGORY'].str.lower() == 'severe weather'].copy()

if 'CAUSE.CATEGORY.DETAIL' in weather_df.columns:
    # Fill missing details with 'Unknown' to include them in the visualization
    weather_df['CAUSE.CATEGORY.DETAIL'] = weather_df['CAUSE.CATEGORY.DETAIL'].fillna('Unknown')

    # Select top 10 most frequent detail categories for clarity
    top_details = weather_df['CAUSE.CATEGORY.DETAIL'].value_counts().nlargest(10).index
    weather_df = weather_df[weather_df['CAUSE.CATEGORY.DETAIL'].isin(top_details)]

fig4 = px.box(weather_df,
              x='CAUSE.CATEGORY.DETAIL',
              y='OUTAGE.DURATION',
              title='Outage Duration by Specific Severe Weather Events (Top 10)',
              color='CAUSE.CATEGORY.DETAIL')

fig4.update_layout(xaxis_tickangle=-45)
fig4.show()
```

Assessment of Missingness

This section investigates the nature of missing data in the `OUTAGE.DURATION` column and applies an appropriate imputation strategy based on the findings.

1. Permutation Test: Dependency on Cause Category

To determine if the missingness is **MAR** or **MCAR**, this project tested the relationship between `OUTAGE.DURATION` missingness and `CAUSE.CATEGORY`.

- **Test Statistic**: Total Variation Distance (TVD)
- **Observed TVD**: 0.2658
- **P-value**: 0.0000

Conclusion: Since the P-value is 0.0000, the null hypothesis is rejected. This confirms that the missingness of outage duration is **MAR (Missing at Random)**, as it shows a strong dependency on the cause of the outage.

2. Hierarchical Imputation Strategy

Since the missingness is confirmed as MAR, using a simple global mean would introduce bias. Instead, this process implements a **Hierarchical Imputation** strategy to preserve the distinct characteristics of each cause:

1. **Level 1**: Fill missing values using the **median** of the specific `CAUSE.CATEGORY.DETAIL`.
2. **Level 2**: For remaining NaNs, use the **median** of the broader `CAUSE.CATEGORY`.
3. **Level 3**: Any final remains are filled with the **global median** to ensure a complete dataset for subsequent modeling.

In [4]:
```
# --- Permutation Test for Missingness in OUTAGE.DURATION by CAUSE.CATEGORY ---
df['is_duration_missing'] = df['OUTAGE.DURATION'].isna()

# TVD Function: Calculate Total Variation Distance between the distribution of missingness across cause categories
def get_tvd(dataframe):
    dist = dataframe.pivot_table(index='CAUSE.CATEGORY', columns='is_duration_missing', aggfunc='size', fill_value=0)
    dist = dist.div(dist.sum(axis=1), axis=0)
    return np.sum(np.abs(dist[True] - dist[True].mean())) / 2

observed_tvd = get_tvd(df)

# Permutation: Shuffle CAUSE.CATEGORY and recalculate TVD to build null distribution
tvds = []
for _ in range(1000):
```

```
    shuffled_causes = df['CAUSE.CATEGORY'].sample(replace=False, frac=1).reset_index(drop=True)
    shuffled_df = df.assign(**{'CAUSE.CATEGORY': shuffled_causes})
    tvds.append(get_tvd(shuffled_df))

# P-value calculation: Proportion of permuted TVDs that are greater than or equal to the observed TVD
p_val = np.mean(np.array(tvds) >= observed_tvd)

print(f"Observed TVD: {observed_tvd:.4f}")
print(f"P-value: {p_val:.4f}")
```

```
Observed TVD: 0.2658
P-value: 0.0000
```

In [5]:

```
# Group the data to compare the count of missing vs. non-missing values across different causes
missing_counts = df.groupby(['is_duration_missing', 'CAUSE.CATEGORY']).size().reset_index(name='count')

# This bar chart visualizes whether certain outage causes are more prone to missing duration data (MAR)
fig_missing = px.bar(
    missing_counts,
    x='CAUSE.CATEGORY',
    y='count',
    color='is_duration_missing',
    barmode='group', # Grouped bars to compare distribution directly
    title='Distribution of Missing vs. Non-Missing Durations by Cause Category',
    labels={'is_duration_missing': 'Is Duration Missing?'}
)

# Rotate x-axis labels for better readability
fig_missing.update_layout(xaxis_tickangle=-45)
fig_missing.show()
```

In [6]:

```
# --- Step 3-2: Hierarchical Imputation based on MAR Conclusion ---

# Group columns to use for hierarchical filling
levels = [['CAUSE.CATEGORY', 'CAUSE.CATEGORY.DETAIL'], ['CAUSE.CATEGORY']]

# Iteratively fill NaNs using group medians
for columns in levels:
    group_median = df.groupby(columns)['OUTAGE.DURATION'].transform('median')
    df['OUTAGE.DURATION'] = df['OUTAGE.DURATION'].fillna(group_median)

# Final fallback: use global median for any remaining gaps
df['OUTAGE.DURATION'] = df['OUTAGE.DURATION'].fillna(df['OUTAGE.DURATION'].median())

# Verify restoration
print(f"Missing Values remaining: {df['OUTAGE.DURATION'].isna().sum()}")
```

```
Missing Values remaining: 0
```

Hypothesis Testing

This section performs a formal statistical test to determine if the presence of **Severe Weather** significantly impacts the restoration time of power outages compared to all other causes.

1. Research Question

Does the recovery time (`OUTAGE.DURATION`) for outages caused by **Severe Weather** differ significantly from the recovery time for outages caused by all other reasons?

2. Hypotheses

- **Null Hypothesis (H_0):** The distribution of outage durations for **Severe Weather** is the same as the distribution for all other causes. Any observed difference is due to random chance.
- **Alternative Hypothesis (H_a):** The average outage duration for **Severe Weather** is significantly **longer** than that for other causes.

3. Test Statistic & Methodology

- **Test Statistic**: Difference in Mean Duration
- **Method**: Permutation Test (1,000 repetitions)
- **Significance Level**: 0.05

4. Results

- **Severe Weather Mean**: 3,846.23 min
- **Other Causes Mean**: 1,501.58 min
- **Observed Difference**: **2,344.65 min**

- **P-value**: **0.0000**

5. Conclusion

Since the **P-value is 0.0000**, which is much lower than the significance level of 0.05, this project **rejects the null hypothesis**. There is strong statistical evidence that outages caused by **Severe Weather** result in significantly longer restoration times—averaging about 39 hours longer—than outages caused by other factors. This confirms that weather is a critical feature for predicting power grid recovery.

In [7]:
```
# --- Data Preparation for Hypothesis Testing ---

# Standardize cause categories by converting to lowercase and stripping whitespace
df['CAUSE_CLEAN'] = df['CAUSE.CATEGORY'].astype(str).str.lower().str.strip()

# Create a binary indicator for 'Severe Weather'
df['IS_SEVERE_WEATHER'] = df['CAUSE_CLEAN'] == 'severe weather'

# Drop missing values in the target variable to ensure a valid analysis
test_df = df.dropna(subset=['OUTAGE.DURATION']).copy()

# Calculate observed means for each group
group_means = test_df.groupby('IS_SEVERE_WEATHER')['OUTAGE.DURATION'].mean()
mean_severe = group_means.get(True, 0)
mean_others = group_means.get(False, 0)
observed_diff = mean_severe - mean_others

# --- Permutation Test ---
# Goal: Determine if the observed difference is statistically significant
n_repetitions = 1000
shuffled_diffs = []
np.random.seed(42)  # Ensure reproducibility

# Convert columns to numpy arrays for faster computation
is_severe_array = test_df['IS_SEVERE_WEATHER'].values
durations_array = test_df['OUTAGE.DURATION'].values

for _ in range(n_repetitions):
    # Randomly shuffle the 'Severe Weather' labels
    shuffled_labels = np.random.permutation(is_severe_array)

    # Calculate means for the shuffled groups
    s_mean_true = durations_array[shuffled_labels].mean() if shuffled_labels.any() else 0
    s_mean_false = durations_array[~shuffled_labels].mean() if (~shuffled_labels).any() else 0
    shuffled_diffs.append(s_mean_true - s_mean_false)

# Calculate the p-value: probability of observing a difference >= observed_diff by chance
p_value_final = np.mean(np.array(shuffled_diffs) >= observed_diff)

# --- Visualization ---
# Map boolean values to descriptive group names for the plot
test_df['Group'] = test_df['IS_SEVERE_WEATHER'].map({True: 'Severe Weather', False: 'Other Causes'})

# Create a notched box plot with fixed dimensions for better readability
fig_hypothesis = px.box(
    test_df,
    x='Group',
    y='OUTAGE.DURATION',
    color='Group',
    notched=True,   # Shows the confidence interval around the median
    width=600,      # Adjusted width for compact view
    height=500,     # Adjusted height for compact view
    title='Comparison of Outage Durations (Log Scale)',
    labels={'OUTAGE.DURATION': 'Duration (min)'},
    category_orders={"Group": ["Severe Weather", "Other Causes"]}
)

# Apply log scale and clean up layout
fig_hypothesis.update_layout(
    yaxis_type="log", # Essential due to heavy right-skewness of the data
    showlegend=False,
    xaxis_title="Outage Cause Group",
    yaxis_title="Duration (min, Log Scale)",
    margin=dict(l=40, r=40, t=60, b=40)
)

fig_hypothesis.show()

# --- Output Results ---
print(f"--- Updated Hypothesis Test Results ---")
print(f"Severe Weather Mean: {mean_severe:.2f} min")
print(f"Other Causes Mean: {mean_others:.2f} min")
print(f"Observed Difference: {observed_diff:.2f} min")
print(f"P-value: {p_value_final:.4f}")
```

```
--- Updated Hypothesis Test Results ---
Severe Weather Mean: 3846.23 min
Other Causes Mean: 1501.58 min
Observed Difference: 2344.65 min
P-value: 0.0000
```

Framing a Prediction Problem

This project aims to build a predictive model that estimates the recovery time of major power outages based on initial event data.

1. Prediction Task

The goal is to predict the **duration of a power outage** (`OUTAGE.DURATION`) at the moment it is reported.

2. Problem Type

Since the target variable is a continuous numerical value (minutes), this is a **Regression** problem.

3. Features and Target Variable

- **Target**: `OUTAGE.DURATION`
- **Features**:
  - **Event Nature**:
    - `CAUSE.CATEGORY` : The primary trigger of the event.
    - `ANOMALY.LEVEL` : Numerical intensity of the climate event.
  - **Temporal & Spatial**:
    - `MONTH` : Captures seasonal restoration challenges (e.g., freezing temperatures).
    - `HOUR` : Represents time-of-day effects on emergency response and visibility for repair crews.
    - `U.S._STATE` : Captures state-specific grid resilience and policy.
  - **Event Scale**:
    - `CUSTOMERS.AFFECTED` : The number of people impacted initially.
    - `DEMAND.LOSS.MW` : The physical power load lost (MW).
  - **Infrastructure & Geography**:
    - `POPDEN_URBAN` : Reflects resource accessibility and restoration priority.
    - `TOTAL.CUSTOMERS` : The overall scale of the state's power infrastructure.
    - `AREAPCT_URBAN` : Indicates the geographical complexity of the affected area.

4. Evaluation Metric

Root Mean Squared Error (RMSE) will be used to evaluate the model.

- **Reasoning**: RMSE penalizes larger prediction errors more heavily than MAE. In the context of power grid management, failing to predict a long-term, high-impact outage is more critical than a slight error in a short-term outage. Therefore, RMSE is the more appropriate metric to ensure the model accounts for high-risk events.

Baseline Model

In this stage, a **Linear Regression** model was established as a baseline to predict the power outage duration (`OUTAGE.DURATION`). To analyze the impact of extreme outliers on model performance, I compared the results of training the model on the **Full Dataset** versus a **Filtered Dataset** (excluding the top 10% outliers).

1. Model Configuration

- **Algorithm**: Linear Regression (Baseline)
- **Features Used**:
  - **Categorical**: `CAUSE.CATEGORY` , `U.S._STATE` , `MONTH` , `HOUR`
  - **Numerical**: `ANOMALY.LEVEL` , `CUSTOMERS.AFFECTED` , `DEMAND.LOSS.MW` , `POPDEN_URBAN` , `TOTAL.CUSTOMERS`
- **Preprocessing Pipeline**:
  - **Numerical Data**: Missing values were imputed with the **Median** and normalized using **Standard Scaling**.
  - **Categorical Data**: Missing values were filled with the constant **'missing'** and transformed using **One-Hot Encoding**.

2. Performance Comparison: Full vs. Filtered Data

The model was evaluated using **RMSE (Root Mean Squared Error)**. To measure the actual predictive power, the results were compared against a **Naive Baseline** (predicting the mean value of the training set).

Scenario	Model RMSE	Naive RMSE (Mean)	Improvement
Full Data (w/ Outliers)	5391.51	5063.43	**-6.48% (Negative)**
Filtered Data (90th Pct)	**1321.29**	**1588.82**	**16.84%**

---

3. Analysis

- **The Impact of Outliers**: When using the full dataset, the model's RMSE (5391.51) was higher than the naive mean prediction (5063.43). This indicates that extreme outliers (outages lasting tens of thousands of minutes) heavily skewed the linear regression line, causing it to lose predictive accuracy for typical cases.
- **Effect of Outlier Removal**: By removing the top 10% extreme values, the RMSE dropped dramatically from 5391.51 to **1321.29**. The model finally achieved a **16.84% improvement** over the naive mean prediction, demonstrating that it began to capture meaningful patterns in the data once the noise from extreme events was reduced.
- **Conclusion**: While outlier removal allowed the model to learn general outage patterns, the 16.84% improvement suggests that the linear model still has limitations. The relationships between infrastructure, climate, and restoration time likely involve non-linear complexities that will be addressed in the Final Model.

In [26]:
```
# --- 0. Feature Extraction (에러 방지용 필수 단계) ---
df['HOUR'] = df['OUTAGE.START'].dt.hour
df['MONTH'] = df['OUTAGE.START'].dt.month

# --- 1. Outlier Removal (90th Percentile) ---
upper_limit = df['OUTAGE.DURATION'].quantile(0.90)
df_filtered = df[(df['OUTAGE.DURATION'] > 0) & (df['OUTAGE.DURATION'] < upper_limit)].copy()

# --- 2. Feature Selection ---
features = [
    'CAUSE.CATEGORY', 'ANOMALY.LEVEL', 'MONTH', 'HOUR',
    'U.S._STATE', 'CUSTOMERS.AFFECTED', 'DEMAND.LOSS.MW',
    'POPDEN_URBAN', 'TOTAL.CUSTOMERS'
]

categorical_features = ['CAUSE.CATEGORY', 'U.S._STATE', 'MONTH', 'HOUR']
for col in categorical_features:
    df_filtered[col] = df_filtered[col].astype(str)

numerical_features = ['ANOMALY.LEVEL', 'CUSTOMERS.AFFECTED', 'DEMAND.LOSS.MW', 'POPDEN_URBAN', 'TOTAL.CUSTOMERS']

X = df_filtered[features]
y = df_filtered['OUTAGE.DURATION']

# --- 3. Data Splitting (70/30) ---
# [Image of train test split process in machine learning]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# --- 4. Preprocessing & Model Pipeline ---
# [Image of Scikit-Learn Pipeline and ColumnTransformer workflow]
num_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

cat_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant', fill_value='missing')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
])

preprocessor = ColumnTransformer(transformers=[
    ('num', num_transformer, numerical_features),
    ('cat', cat_transformer, categorical_features)
])

# Define the Baseline Model (Linear Regression)
baseline_model = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', LinearRegression())
])

# --- 5. Training and Evaluation ---
baseline_model.fit(X_train, y_train)
y_pred = baseline_model.predict(X_test)

# Calculate Metrics
# [Image of Root Mean Square Error RMSE formula and intuition]
rmse_baseline = np.sqrt(mean_squared_error(y_test, y_pred))
y_mean_val = y_train.mean()
naive_rmse = np.sqrt(mean_squared_error(y_test, np.full(len(y_test), y_mean_val)))
improvement = (1 - (rmse_baseline / naive_rmse)) * 100
```

```
print(f"--- Baseline Model Results (Filtered Data) ---")
print(f"90th Percentile Limit: {upper_limit:.2f} min")
print(f"Number of samples used: {len(df_filtered)}")
print(f"Test RMSE: {rmse_baseline:.2f} minutes")
print(f"Improvement over Mean: {improvement:.2f}%")

# --- 6. Visualization: Model vs. Naive RMSE ---
labels = ['Naive (Mean) Prediction', 'Baseline Model (Linear Regression)']
rmses = [naive_rmse, rmse_baseline]

fig = go.Figure(data=[
    go.Bar(
        x=labels,
        y=rmses,
        text=[f"{v:.2f}" for v in rmses],
        textposition='auto',
        marker_color=['#B2CFEE', '#636EFA'] # Naive는 연한 색, Model은 강조 색
    )
])

fig.update_layout(
    title='Baseline Model Performance: RMSE Comparison',
    yaxis_title='RMSE (Minutes)',
    template='plotly_white',
    width=600, height=500
)

fig.show()
```

```
--- Baseline Model Results (Filtered Data) ---
90th Percentile Limit: 6891.80 min
Number of samples used: 1380
Test RMSE: 1321.29 minutes
Improvement over Mean: 16.84%
```

Final Model

In this final stage, I developed an optimized **Random Forest Regressor** to capture the non-linear relationships and complex interactions between features that the linear baseline model could not account for. To ensure the best performance, I employed **GridSearchCV** for hyperparameter tuning.

1. Model Configuration & Hyperparameter Tuning

- **Algorithm**: Random Forest Regressor
- **Tuning Method**: GridSearchCV (3-fold Cross-Validation)
- **Hyperparameters Optimized**:
  - `n_estimators` : Number of trees in the forest.
  - `max_depth` : Maximum depth of each tree to prevent overfitting.
  - `min_samples_split` : Minimum samples required to split an internal node.
- **Best Parameters Found**:
  - `max_depth` : 20
  - `min_samples_split` : 10
  - `n_estimators` : 300

2. Final Performance Evaluation

The final model was tested on the same filtered dataset (90th percentile) to ensure a fair comparison with the baseline.

Metric	Naive Mean Prediction	Baseline (Linear Regression)	Final Model (Random Forest)
Test RMSE (Minutes)	1588.82	1321.29	**1260.02**
Improvement over Mean	-	16.84%	**20.69%**

3. Analysis and Conclusion

- **Performance Gain**: The Final Model achieved a **20.69% improvement** over the naive mean prediction and a **4.64% further reduction in RMSE** compared to the Linear Baseline. This confirms that the Random Forest algorithm successfully captured non-linear patterns, such as the interaction between specific climate regions and cause categories.
- **Model Stability**: By using a `max_depth` of 20 and `min_samples_split` of 10, the model maintains a balance between complexity and generalization, avoiding overfitting to specific noise in the outage data.
- **Final Takeaway**: Starting from a model that was worse than the average (-6.48% improvement with outliers), the project successfully progressed through **data cleaning (90th percentile filtering)** and **algorithm optimization (Random Forest)** to reach a meaningful predictive accuracy of ~1,260 minutes.

In [29]:
```
# 1. Define the Final Model Pipeline
final_model_pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('regressor', RandomForestRegressor(random_state=42))
])

# 2. Define the Hyperparameter Grid for Tuning
param_grid = {
    'regressor__n_estimators': [100, 200, 300],
    'regressor__max_depth': [10, 20, None],
    'regressor__min_samples_split': [2, 5, 10]
}

# 3. Setup GridSearchCV
grid_search = GridSearchCV(
    final_model_pipeline,
    param_grid,
    cv=3,
    scoring='neg_root_mean_squared_error',
    verbose=1,
    n_jobs=-1
)

# 4. Train the Final Model on the Filtered Data
grid_search.fit(X_train, y_train)

# 5. Evaluate the Best Model
best_model = grid_search.best_estimator_
y_final_pred = best_model.predict(X_test)
final_rmse = np.sqrt(mean_squared_error(y_test, y_final_pred))

# --- Fixed Part: Recalculate necessary values for comparison ---
# 6. Calculate Baselines again to avoid NameError
y_train_mean = y_train.mean()
current_naive_rmse = np.sqrt(mean_squared_error(y_test, np.full(len(y_test), y_train_mean)))
# Baseline model (Linear Regression) RMSE from Step 6 was 1321.29
baseline_rmse = 1321.29

# 7. Calculate Final Improvements
final_improvement_over_mean = (1 - (final_rmse / current_naive_rmse)) * 100
final_improvement_over_baseline = (1 - (final_rmse / baseline_rmse)) * 100

print(f"--- Final Model (Random Forest) Results ---")
print(f"Best Parameters: {grid_search.best_params_}")
print(f"Final Test RMSE: {final_rmse:.2f} minutes")
print(f"Improvement over Naive Mean: {final_improvement_over_mean:.2f}%")
print(f"Improvement over Baseline (Linear): {final_improvement_over_baseline:.2f}%")
```

```
Fitting 3 folds for each of 27 candidates, totalling 81 fits
--- Final Model (Random Forest) Results ---
Best Parameters: {'regressor__max_depth': 20, 'regressor__min_samples_split': 10, 'regressor__n_estimators': 300}
Final Test RMSE: 1260.02 minutes
Improvement over Naive Mean: 20.69%
Improvement over Baseline (Linear): 4.64%
```

Fairness Analysis

In this final step, I conducted a fairness analysis to determine if the model's performance significantly differs across different demographic groups. Specifically, I compared the prediction accuracy between **Urban** and **Rural** areas based on population density (`POPDEN_URBAN`).

1. Methodology

- **Group Definition**: The dataset was split into two groups using the median of `POPDEN_URBAN`.
  - **Urban**: Areas with population density above or equal to the median.
  - **Rural**: Areas with population density below the median.
- **Evaluation Metric**: Root Mean Squared Error (RMSE).
- **Parity Metric**: RMSE Ratio ($RMSE_{Urban}/RMSE_{Rural}$). A ratio near 1.0 indicates parity (fairness).
- **Statistical Significance**: A **Permutation Test** (500 iterations) was used to calculate the **p-value**, testing the null hypothesis that there is no significant difference in RMSE between the two groups.

2. Results

- **Urban Group RMSE**: 864.86 minutes
- **Rural Group RMSE**: 979.64 minutes
- **Fairness Ratio (Urban/Rural)**: **0.8828**

- **P-value**: **0.0740**

3. Conclusion & Interpretation

- **Statistical Insignificance**: The p-value of **0.0740** is greater than the standard significance level of **0.05**. This suggests that the observed difference in RMSE between Urban and Rural areas is **not statistically significant** and could likely be due to random chance.
- **Fairness Confirmed**: The Fairness Ratio of **0.8828** falls well within the acceptable range (0.8 to 1.25). This indicates that the model is **fair** and does not systematically discriminate against any specific group based on their population density.
- **Final Takeaway**: By validating both the technical parity and statistical insignificance of the error difference, I have ensured that the power outage restoration time estimates are equitable. This model provides a reliable and unbiased service to both urban and rural communities.

In [34]:
```
# 1. Split data into two groups based on Population Density
median_popden = df_filtered['POPDEN_URBAN'].median()
group_urban = df_filtered[df_filtered['POPDEN_URBAN'] >= median_popden]
group_rural = df_filtered[df_filtered['POPDEN_URBAN'] < median_popden]

# 2. Function to evaluate RMSE for each group
def evaluate_fairness(group, model, title):
    X_group = group[features]
    y_group = group['OUTAGE.DURATION']
    y_pred = model.predict(X_group)
    rmse = np.sqrt(mean_squared_error(y_group, y_pred))
    return rmse

# 3. Calculate RMSE for each group using the best model
rmse_urban = evaluate_fairness(group_urban, best_model, "Urban")
rmse_rural = evaluate_fairness(group_rural, best_model, "Rural")

# 4. Calculate Parity Ratio (RMSE Ratio)
# A ratio close to 1.0 means the model is fair between groups.
parity_ratio = rmse_urban / rmse_rural

print(f"--- Fairness Analysis Results ---")
print(f"Urban Group RMSE: {rmse_urban:.2f} minutes")
print(f"Rural Group RMSE: {rmse_rural:.2f} minutes")
print(f"Fairness Ratio (Urban/Rural): {parity_ratio:.4f}")

# 5. Permutation Test for Statistical Significance
# Does this difference happen by chance?
diff_obs = abs(rmse_urban - rmse_rural)
met_diffs = []

# Shuffle group labels 500 times
all_data = df_filtered.copy()
for _ in range(500):
    shuffled = all_data['POPDEN_URBAN'].sample(replace=False, frac=1).reset_index(drop=True)
    all_data['shuffled_pop'] = shuffled
    g1 = all_data[all_data['shuffled_pop'] >= median_popden]
    g2 = all_data[all_data['shuffled_pop'] < median_popden]

    r1 = evaluate_fairness(g1, best_model, "G1")
    r2 = evaluate_fairness(g2, best_model, "G2")
    met_diffs.append(abs(r1 - r2))

p_value = np.mean(np.array(met_diffs) >= diff_obs)
print(f"P-value: {p_value:.4f}")
```

```
--- Fairness Analysis Results ---
Urban Group RMSE: 864.86 minutes
Rural Group RMSE: 979.64 minutes
Fairness Ratio (Urban/Rural): 0.8828
P-value: 0.0740
```